

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Τεχνολογίες Εφαρμογών Διαδικτύου 2026

Ονόματα Γιώργος Δέλγας — Γιώργος Παλαιοκώστας

Αριθμοί Μητρώου 1115202200036 — 1115202200126

Περιεχόμενα

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων	1
1. Εισαγωγή	3
2. Αρχιτεκτονική Συστήματος	4
2.1 Επισκόπηση	4
2.2 Τεχνολογικό Πλαίσιο	4
2.3 Δομή Αρχείων Backend	5
3. Frontend Αρχιτεκτονική	5
3.1 Επισκόπηση	5
3.2 Δομή Αρχείων Frontend	5
3.3 Δρομολόγηση και Lazy Loading	6
3.4 Guards — Έλεγχος Πρόσβασης βάσει Ρόλου	6
3.5 Resolvers — Προφόρτωση Δεδομένων	6
3.6 Interceptors — HTTP Παρεμβολείς	7
3.7 Υπηρεσίες Δεδομένων	7
3.8 Καθολικές Υπηρεσίες UI	8
4. Σχεδιασμός Βάσης Δεδομένων	8
4.1 Εργαλεία και Migrations	8
4.2 Πίνακες	9
4.3 Αποφάσεις Σχεδιασμού	9
5. Αυθεντικοποίηση και Εξουσιοδότηση	10
5.1 JWT (JSON Web Tokens)	10
5.2 Middleware Αυθεντικοποίησης	10
5.3 Role Middleware	10

5.4 Ασφαλής Αποθήκευση Κωδικών	10
6. API Endpoints	10
6.1 Αυθεντικοποίηση (/api/auth)	11
6.2 Χρήστες (/api/users)	11
6.3 Εκδηλώσεις (/api/events)	11
6.4 Κρατήσεις (/api/bookings)	11
6.5 Μηνύματα (/api/messages)	11
6.6 Export (/api/export)	12
7. Επιχειρησιακή Λογική	12
7.1 Κρατήσεις — Αποφυγή Υπερκράτησης	12
7.2 Αυτόματη Ολοκλήρωση Εκδηλώσεων	12
7.3 Επικύρωση Εκδηλώσεων	12
7.4 XML Export	13
7.5 Παρακολούθηση Προβολών	13
8. Αλγόριθμος Συστάσεων	13
8.1 Επισκόπηση	13
8.2 Δεδομένα Εκπαίδευσης	13
8.3 Διαχωρισμός Χώρων ID	14
8.4 Πρόγραμμα Εκπαίδευσης	14
8.5 Προτάσεις για Νέους Χρήστες	14
9. Ασφάλεια	14
9.1 HTTPS	14
9.2 Helmet	15
9.3 CORS	15
9.4 Rate Limiting	15
9.5 Εξουσιοδότηση σε Επίπεδο Πόρων	15
9.6 Απόκρυψη Ευαίσθητων Δεδομένων	15
10. Εγκατάσταση και Εκτέλεση	15
10.1 Προαπαιτούμενα	15
10.2 Αυτόματη Εγκατάσταση	15
10.3 Χειροκίνητη Εγκατάσταση	16
10.4 Εκτέλεση	16
10.5 Αποδοχή SSL Πιστοποιητικού	16

10.6 Μεταβλητές Περιβάλλοντος (.env).....	17
11. Επίλογος	17
11.1 Σύνοψη Υλοποίησης	17
11.2 Δυσκολίες και Αντιμετώπισή τους.....	17

1. Εισαγωγή

Στόχος της εργασίας είναι η ανάπτυξη μιας πλήρους εφαρμογής διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων εισιτηρίων στον Παγκόσμιο Ιστό. Η εφαρμογή υλοποιεί τέσσερις ρόλους χρηστών (Διαχειριστής, Διοργανωτής, Συμμετέχων, Επισκέπτης) και παρέχει πλήρη υποδομή για δημιουργία εκδηλώσεων, κράτηση εισιτηρίων, ανταλλαγή μηνυμάτων και εξατομικευμένες συστάσεις.

Το παρόν κείμενο περιγράφει τόσο το **νωτιαίο άκρο (backend)** της εφαρμογής — τον REST API server που χειρίζεται την επιχειρησιακή λογική, επικοινωνεί με τη βάση δεδομένων και εξυπηρετεί τα αιτήματα του frontend — όσο και το **μπροστινό άκρο (frontend)** — την Angular SPA που παρέχει τη διεπαφή χρήστη.

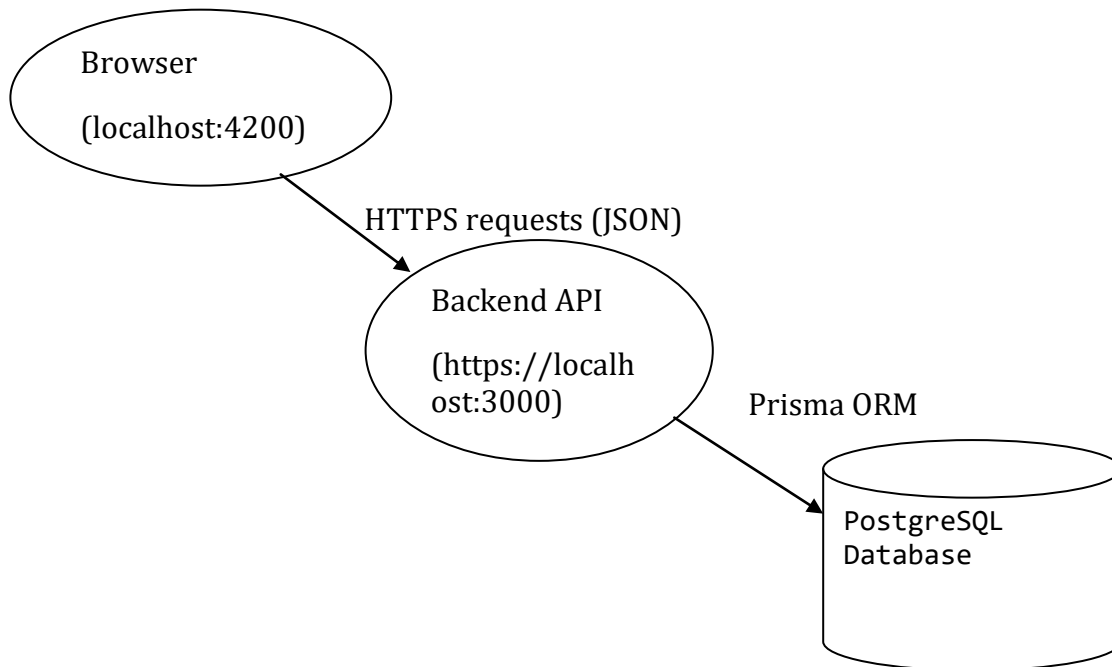
Τα κεφάλαια που ακολουθούν καλύπτουν:

- Την αρχιτεκτονική και τις τεχνολογίες που χρησιμοποιήθηκαν
- Την αρχιτεκτονική του frontend, τη δρομολόγηση, guards, resolvers και interceptors
- Τον σχεδιασμό της βάσης δεδομένων και τις αποφάσεις που λήφθηκαν
- Τον μηχανισμό αυθεντικοποίησης και εξουσιοδότησης
- Τα διαθέσιμα API endpoints
- Την υλοποίηση της επιχειρησιακής λογικής (κρατήσεις, εκδηλώσεις, μηνύματα)
- Τον αλγόριθμο συστάσεων Biased Matrix Factorization
- Τα μέτρα ασφαλείας
- Οδηγίες εγκατάστασης και εκτέλεσης

2. Αρχιτεκτονική Συστήματος

2.1 Επισκόπηση

Το σύστημα ακολουθεί αρχιτεκτονική **client-server** με πλήρη διαχωρισμό frontend και backend



Το backend εκτελείται ως **HTTPS server** στη θύρα 3000 και εκθέτει ένα REST API. Το frontend (Angular) επικοινωνεί αποκλειστικά μέσω HTTPS αιτημάτων JSON.

2.2 Τεχνολογικό Πλαίσιο

Επίπεδο	Τεχνολογία	Αιτιολόγηση
Runtime	Node.js v20	Ασύγχρονο μοντέλο εκτέλεσης, ευρύ οικοσύστημα
Framework	Express 4	Ελαφρύ, ευέλικτο
Γλώσσα	TypeScript 5	Στατικοί τύποι, ασφαλέστερος κώδικας
ORM	Prisma 5	Πλήρως τυποποιημένη πρόσβαση στη ΒΔ, versioned migrations
Βάση Δεδομένων	PostgreSQL 14+	Σχεσιακά δεδομένα, ισχυρή συνέπεια, ACID εγγυήσεις
Αυθεντικοποίηση	JWT + bcryptjs	Stateless αυθεντικοποίηση, ασφαλής αποθήκευση κωδικών
SSL	Node.js https + openssl	Κρυπτογράφηση όλης της επικοινωνίας

2.3 Δομή Αρχείων Backend

```
backend/
├── prisma/
│   ├── schema.prisma      # Ορισμός σχήματος ΒΔ
│   ├── seed.ts            # Αρχική εισαγωγή admin χρήστη
│   └── migrations/        # Versioned SQL migrations
├── src/
│   ├── app.ts             # Entry point – Express setup, HTTPS server
│   ├── config/
│   │   └── prisma.ts      # Singleton PrismaClient
│   ├── controllers/       # HTTP request handlers (λεπτό επίπεδο)
│   ├── services/          # Επιχειρησιακή λογική
│   ├── routes/            # Express router definitions
│   ├── middleware/        # Auth, role checking
│   ├── models/            # Interfaces/types
│   ├── utils/             # JWT utilities, XML generation
│   └── certs/             # SSL πιστοποιητικά
```

3. Frontend Αρχιτεκτονική

3.1 Επισκόπηση

Το frontend υλοποιήθηκε με **Angular 21** ως Single Page Application (SPA). Επικοινωνεί αποκλειστικά με το backend μέσω HTTPS αιτημάτων JSON. Χρησιμοποιεί standalone components χωρίς NgModule, lazy loading ανά feature, και **Angular Signals** για αντιδραστική διαχείριση κατάστασης χωρίς εξωτερική βιβλιοθήκη.

3.2 Δομή Αρχείων Frontend

```
frontend/src/app/
├── app.component.ts       # Root component
├── app.routes.ts          # Κεντρική ρύθμιση δρομολόγησης
├── core/
│   ├── guards/
│   │   ├── auth.guard.ts  # Έλεγχος αυθεντικοποίησης
│   │   ├── role.guard.ts  # Έλεγχος ρόλου χρήστη
│   │   └── event-list-filters.guard.ts # Επικύρωση query παραμέτρων
│   ├── interceptors/
│   │   ├── jwt.interceptor.ts # Προσθήκη Bearer token
│   │   └── loading.interceptor.ts # Καθολικός δείκτης φόρτωσης
│   ├── resolvers/        # Προφόρτωση δεδομένων ανά route
│   └── services/         # AuthService, EventService, κ.ά.
├── features/
│   ├── auth/             # Login, Register
│   ├── events/           # Λίστα, λεπτομέρειες, φόρμα εκδήλωσης
│   ├── bookings/        # Κρατήσεις χρήστη, φόρμα κράτησης
│   └── messaging/        # Εισερχόμενα, απεσταλμένα, σύνθεση
```

		admin/	# Διαχείριση χρηστών, export
		welcome/	# Αρχική σελίδα
		shared/	
		components/	# Navbar, Toast, Modal, Loading, Map
		models/	# TypeScript interfaces (User, Event, Booking)

3.3 Δρομολόγηση και Lazy Loading

Κάθε feature ορίζει τα routes του σε ξεχωριστό αρχείο `.routes.ts` και φορτώνεται δυναμικά μέσω `loadChildren`. Αυτό επιτρέπει **code splitting**: ο κώδικας κάθε feature φορτώνεται μόνο όταν ο χρήστης πλοηγηθεί σε αυτό, μειώνοντας τον χρόνο αρχικής φόρτωσης.

Επιπλέον χαρακτηριστικά της δρομολόγησης:

`withComponentInputBinding()`: Τα δεδομένα του route (resolver data, params, query params) μεταδίδονται αυτόματα ως `input()` στο component, αποφεύγοντας το `ActivatedRoute`.

`withInMemoryScrolling()`: Αποκατάσταση θέσης scrolling κατά την πλοήγηση στο ιστορικό.

`runGuardsAndResolvers: 'paramsOrQueryParamsChange'`: Επαναεκτέλεση guards και resolvers σε κάθε αλλαγή query παραμέτρων (π.χ. εφαρμογή φίλτρων αναζήτησης).

3.4 Guards — Έλεγχος Πρόσβασης βάσει Ρόλου

Τρία guards ελέγχουν την πρόσβαση πριν την ενεργοποίηση κάθε route:

AuthGuard: Ελέγχει αν ο χρήστης είναι συνδεδεμένος. Αν δεν είναι, ανακατευθύνει στο `/login`. Χρησιμοποιείται για routes που απαιτούν αυθεντικοποίηση (π.χ. `/messages`, `/bookings`).

RoleGuard: Ελέγχει αν ο ρόλος του συνδεδεμένου χρήστη περιλαμβάνεται στη λίστα `route.data['roles']`. Αν δεν πληροί τον έλεγχο, ανακατευθύνει στο `/events`. Οι τέσσερις υποστηριζόμενοι ρόλοι είναι: `Admin`, `Organizer`, `Participant` και `Undefined` (μη συνδεδεμένος επισκέπτης). Παραδείγματα χρήσης: το `/admin` απαιτεί `Admin`, το `/events/manage` απαιτεί `Organizer`.

eventListFiltersGuard: Επικυρώνει τις query παραμέτρους φίλτρων (κατηγορία, εύρος ημερομηνιών, εύρος τιμής, σελίδα) πριν από την απόδοση της λίστας εκδηλώσεων. Αν εντοπιστεί μη έγκυρη τιμή, ανακατευθύνει με καθαρισμένες παραμέτρους, αποτρέποντας εκτεθειμένα σφάλματα από κακοδιαμορφωμένα URLs.

3.5 Resolvers — Προφόρτωση Δεδομένων

Όλα τα components που απαιτούν δεδομένα από το API τα λαμβάνουν **πριν από την ενεργοποίησή τους** μέσω resolvers. Αυτό εγγυάται ότι το component φορτώνει πάντα με τα απαιτούμενα δεδομένα έτοιμα, αποφεύγοντας ενδιάμεσες καταστάσεις φόρτωσης μέσα

στο template. Σε περίπτωση σφάλματος δικτύου ή αδύνατης ανάκτησης, εμφανίζεται toast ειδοποίηση και επιστρέφεται EMPTY, το οποίο ακυρώνει την πλοήγηση.

Resolver Δεδομένα

Events : Λίστα εκδηλώσεων (δημόσιων ή του διοργανωτή) με υποστήριξη φίλτρων και σελιδοποίησης

event : Λεπτομέρειες μίας εκδήλωσης βάσει ID

myBookings : Κρατήσεις του συνδεδεμένου χρήστη

bookings : Κρατήσεις εκδήλωσης (για διοργανωτή/admin)

messages : Εισερχόμενα και απεσταλμένα μηνύματα παράλληλα με forkJoin

recommendedEvents : Εξατομικευμένες συστάσεις εκδηλώσεων

users : Λίστα χρηστών (για admin)

userDetail : Λεπτομέρειες χρήστη (για admin)

role : Ρόλος του συνδεδεμένου χρήστη

3.6 Interceptors — HTTP Παρεμβολείς

Δύο interceptors καταχωρούνται καθολικά στο main.ts και εφαρμόζονται αυτόματα σε κάθε αίτημα:

JwtInterceptor: Προσθέτει το JWT token ως Authorization: Bearer <token> header σε όλα τα εξερχόμενα αιτήματα. Αν το αίτημα επιστρέψει HTTP 401 (token ληγμένο ή άκυρο), αποσυνδέει αυτόματα τον χρήστη, καθαρίζει το localStorage και ανακατευθύνει στο /login.

LoadingInterceptor: Παρακολουθεί τον αριθμό ενεργών HTTP αιτημάτων με μετρητή increment/decrement. Ενεργοποιεί τον καθολικό κινούμενο δείκτη φόρτωσης όσο υπάρχει τουλάχιστον ένα αίτημα σε εκκρεμότητα. Αιτήματα που φέρουν το context token SKIP_LOADING (π.χ. περιοδικό polling αναγνωσμένων μηνυμάτων) εξαιρούνται από τον μηχανισμό και δεν προκαλούν εμφάνιση του spinner.

3.7 Υπηρεσίες Δεδομένων

Όλες οι υπηρεσίες ορίζονται ως singletons (providedIn: 'root'). Τα HTTP αιτήματα επιστρέφονται ως RxJS Observables και καταναλώνονται κατά κανόνα μέσω των resolvers.

AuthService: Διαχειρίζεται την κατάσταση αυθεντικοποίησης με Angular Signals. Το `currentUser` signal (readonly) αποτελεί τη μοναδική πηγή αλήθειας για τον συνδεδεμένο χρήστη. Τα `isLoggedIn`, `userRole`, `isAdmin`, `isOrganizer`, `isParticipant` υπολογίζονται αυτόματα ως **computed signals** και χρησιμοποιούνται απευθείας στα templates. Κατά την εκκίνηση της εφαρμογής, η κατάσταση αρχικοποιείται από το `localStorage` για να διατηρηθεί η σύνδεση μετά από ανανέωση σελίδας.

EventService, BookingService, MessageService, UserService: Ενθυλακώνουν τις HTTP κλήσεις στο backend και κατασκευάζουν τα `HttpParams` για φίλτρα και σελιδοποίηση. Το `MessageService` διατηρεί επίσης signal για τον αριθμό αδιάβαστων μηνυμάτων, που ανανεώνεται περιοδικά από το `NavbarComponent` κάθε 30 δευτερόλεπτα χωρίς να ενεργοποιεί τον καθολικό spinner (μέσω `SKIP_LOADING`).

3.8 Καθολικές Υπηρεσίες UI

LoadingService: Καθολικός δείκτης φόρτωσης υλοποιημένος με μετρητή αιτημάτων (`BehaviorSubject`) και signal για το UI. Το `LoadingComponent` εμφανίζεται ως overlay σε όλη την εφαρμογή όταν υπάρχουν εκκρεμή αιτήματα.

ToastService: Υπηρεσία ειδοποιήσεων τύπου “toast” που εμφανίζει μηνύματα επιτυχίας, σφάλματος ή προειδοποίησης. Κάθε ειδοποίηση απορρίπτεται αυτόματα μετά από 4 δευτερόλεπτα με animation εξόδου. Χρησιμοποιείται σε `resolvers` για ειδοποιήσεις σφάλματος και σε `components` για επιβεβαίωση επιτυχημένων ενεργειών.

ModalService: Υπηρεσία επιβεβαιωτικών παραθύρων βασισμένη σε `Promise`. Χρησιμοποιείται για ενέργειες που απαιτούν επιβεβαίωση από τον χρήστη (π.χ. ακύρωση κράτησης, διαγραφή εκδήλωσης, αναστολή χρήστη). Υποστηρίζει προσαρμοσμένα templates ή απλά διαλογικά παράθυρα με ρυθμιζόμενα κουμπιά επιβεβαίωσης και ακύρωσης.

4. Σχεδιασμός Βάσης Δεδομένων

4.1 Εργαλεία και Migrations

Η βάση δεδομένων διαχειρίζεται αποκλειστικά μέσω **Prisma Migrate**. Κάθε αλλαγή στο σχήμα εφαρμόζεται μέσω versioned migration αρχείων που αποθηκεύονται στο `prisma/migrations/`.

4.2 Πίνακες

Η βάση αποτελείται από 8 πίνακες:

Πίνακας	Περιγραφή
users	Λογαριασμοί χρηστών με ρόλο και κατάσταση έγκρισης
events	Εκδηλώσεις με τοποθεσία, ημερομηνίες, χωρητικότητα, κατάσταση
event_categories	Κατηγορίες ανά εκδήλωση (πολλές ανά εκδήλωση)
event_photos	URLs φωτογραφιών ανά εκδήλωση
ticket_types	Τύποι εισιτηρίων ανά εκδήλωση (όνομα, τιμή, ποσότητα, διαθέσιμα)
bookings	Κρατήσεις εισιτηρίων που συνδέουν χρήστες με εκδηλώσεις
messages	Εσωτερικά μηνύματα μεταξύ χρηστών
event_views	Παρακολούθηση προβολών εκδηλώσεων ανά χρήστη (για συστάσεις)

4.3 Αποφάσεις Σχεδιασμού

Χωριστοί πίνακες για κατηγορίες και φωτογραφίες: Αντί να αποθηκεύονται ως arrays ή JSON, κάθε κατηγορία και φωτογραφία αποθηκεύεται ως ξεχωριστή εγγραφή με foreign key στην εκδήλωση. Αυτό επιτρέπει εύκολη αναζήτηση βάσει κατηγορίας και αποφεύγει τα προβλήματα κανονικοποίησης.

Πεδίο available στο ticket_types: Τηρείται ξεχωριστό μετρητή διαθέσιμων εισιτηρίων που ενημερώνεται ατομικά με κάθε κράτηση. Αυτό επιτρέπει γρήγορο έλεγχο διαθεσιμότητας χωρίς άθροισμα από τον πίνακα κρατήσεων σε κάθε αίτημα.

Soft delete για μηνύματα: Ο πίνακας messages διαθέτει πεδία deleted_by_sender και deleted_by_receiver. Η εγγραφή διαγράφεται φυσικά (hard delete) μόνο όταν και οι δύο πλευρές το έχουν διαγράψει. Έτσι διατηρείται το ιστορικό για τον άλλο χρήστη.

Πίνακας event_views: Χρησιμοποιεί σύνθετο primary key (user_id, event_id) με UPSERT λογική — κάθε συνδυασμός χρήστη-εκδήλωσης αποθηκεύεται μία φορά, ενημερώνοντας μόνο το viewed_at. Αυτό αποτρέπει τη διόγκωση του πίνακα από πολλαπλές επισκέψεις.

Μοναδικότητα AFM: Το πεδίο afm έχει UNIQUE constraint στη βάση. Επιπλέον, η εφαρμογή ελέγχει για διπλότυπο AFM σε επίπεδο service πριν από την εισαγωγή, ώστε να επιστρέφει κατανοητό μήνυμα σφάλματος αντί για γενικό database error.

Νέοι χρήστες ξεκινούν ως pending και εγκρίνονται από τον Διαχειριστή. Εγκεκριμένοι χρήστες μπορούν να τεθούν σε suspended χωρίς διαγραφή του λογαριασμού τους.

5. Αυθεντικοποίηση και Εξουσιοδότηση

5.1 JWT (JSON Web Tokens)

Κατά την επιτυχή σύνδεση, ο server δημιουργεί ένα **JWT** που περιέχει: - id: το ID του χρήστη στη βάση - username: το όνομα χρήστη - role: ο ρόλος (admin, organizer, participant) - status: η κατάσταση του λογαριασμού

Το token υπογράφεται με το JWT_SECRET από το .env και έχει διάρκεια ζωής 24 ωρών. Κάθε αίτημα προς προστατευμένα endpoints πρέπει να φέρει το token στο Authorization: Bearer <token> header.

5.2 Middleware Αυθεντικοποίησης

Υλοποιούνται δύο middleware:

Authenticate : Χρησιμοποιείται για endpoints που απαιτούν σύνδεση. Επαληθεύει το JWT και ελέγχει **live** την κατάσταση του χρήστη από τη βάση δεδομένων. Αυτό σημαίνει ότι εάν ο Διαχειριστής αναστείλει έναν χρήστη, ο αποκλεισμός ισχύει άμεσα για το επόμενο αίτημά του, ακόμα και αν το JWT δεν έχει λήξει.

optionalAuthenticate: Χρησιμοποιείται για public endpoints (π.χ. αναζήτηση εκδηλώσεων) όπου ο χρήστης μπορεί να είναι είτε συνδεδεμένος είτε επισκέπτης. Εάν υπάρχει έγκυρο token, προσθέτει τα στοιχεία χρήστη στο request - αλλιώς συνεχίζει χωρίς σφάλμα.

5.3 Role Middleware

Το requireRole(...roles) middleware ελέγχει αν ο συνδεδεμένος χρήστης έχει έναν από τους επιτρεπόμενους ρόλους. Χρησιμοποιείται σε συνδυασμό με authenticate:

```
router.patch('/:id/approve', authenticate, requireRole('admin'), approveUser);
router.post('/', authenticate, requireRole('organizer'), createEvent);
```

5.4 Ασφαλής Αποθήκευση Κωδικών

Οι κωδικοί αποθηκεύονται ως **bcrypt hashes** με salt factor 10. Κατά τη σύνδεση, εφαρμόζεται **constant-time σύγκριση** ακόμα και για μη υπαρκτούς χρήστες (με ψεύτικο hash), αποτρέποντας timing attacks που θα αποκάλυπταν αν ένα username υπάρχει στη βάση.

6. API Endpoints

Όλα τα endpoints βρίσκονται κάτω από το prefix /api/.

6.1 Αυθεντικοποίηση (/api/auth)

Μέθοδος	URL	Περιγραφή
POST	/register	Εγγραφή νέου χρήστη (επιστρέφει μήνυμα αναμονής έγκρισης)
POST	/login	Σύνδεση, επιστρέφει JWT token και δεδομένα χρήστη

Και τα δύο endpoints προστατεύονται από **rate limiting**: μέγιστο 20 αιτήματα ανά 15 λεπτά ανά IP, για αποτροπή brute-force επιθέσεων.

6.2 Χρήστες (/api/users)

Μέθοδος	URL	Εξουσιοδότηση	Περιγραφή
GET	/	Admin	Λίστα χρηστών (φιλτράρισμα βάσει ρόλου/κατάστασης)
GET	/:id	Οποιοσδήποτε	Προφίλ χρήστη (AFM κρυφό εκτός αν ο ίδιος ή admin)
PATCH	/:id/approve	Admin	Έγκριση χρήστη σε pending κατάσταση
PATCH	/:id/reject	Admin	Απόρριψη χρήστη σε pending κατάσταση
PATCH	/:id/suspend	Admin	Αναστολή χρήστη σε approved κατάσταση

6.3 Εκδηλώσεις (/api/events)

Μέθοδος	URL	Εξουσιοδότηση	Περιγραφή
GET	/	Public	Αναζήτηση/λίστα δημοσιευμένων εκδηλώσεων
GET	/:id	Public	Λεπτομέρειες εκδήλωσης
GET	/my	Organizer	Εκδηλώσεις του διοργανωτή
GET	/recommendations	Participant	Εξατομικευμένες συστάσεις
POST	/	Organizer	Δημιουργία εκδήλωσης
PUT	/:id	Organizer	Επεξεργασία εκδήλωσης
PATCH	/:id/publish	Organizer	Δημοσίευση draft εκδήλωσης
PATCH	/:id/cancel	Organizer	Ακύρωση εκδήλωσης
DELETE	/:id	Organizer	Διαγραφή draft εκδήλωσης

6.4 Κρατήσεις (/api/bookings)

Μέθοδος	URL	Εξουσιοδότηση	Περιγραφή
POST	/	Participant	Κράτηση εισιτηρίων
GET	/my	Participant	Οι κρατήσεις του χρήστη
GET	/event/:id	Organizer/Admin	Κρατήσεις για συγκεκριμένη εκδήλωση
PATCH	/:id/cancel	Participant	Ακύρωση κράτησης

6.5 Μηνύματα (/api/messages)

Μέθοδος	URL	Περιγραφή
GET	/inbox	Εισερχόμενα (σελιδοποιημένα, μη αναγνωσμένα πρώτα)
GET	/sent	Απεσταλμένα (σελιδοποιημένα)
GET	/unread-count	Αριθμός μη αναγνωσμένων
POST	/	Αποστολή μηνύματος
PATCH	/:id/read	Σήμανση ως αναγνωσμένο

Μέθοδος	URL	Περιγραφή
DELETE	<code>/:id</code>	Διαγραφή μηνύματος

6.6 Export (/api/export)

Μέθοδος	URL	Περιγραφή
GET	<code>/xml</code>	Λήψη όλων των εκδηλώσεων σε XML (με inline DTD)
GET	<code>/json</code>	Λήψη όλων των εκδηλώσεων σε JSON

7. Επιχειρησιακή Λογική

7.1 Κρατήσεις — Αποφυγή Υπερκράτησης

Η κράτηση εισιτηρίων είναι η πιο κρίσιμη λειτουργία ως προς τη συνέπεια δεδομένων. Εάν δύο χρήστες κάνουν ταυτόχρονα κράτηση για τα τελευταία διαθέσιμα εισιτήρια, χωρίς κατάλληλο συγχρονισμό θα μπορούσαν και οι δύο να επιτύχουν — οδηγώντας σε υπερκράτηση.

Λύση: Η κράτηση εκτελείται μέσα σε **database transaction με row-level locking** (SELECT ... FOR UPDATE). Αυτό κλειδώνει την εγγραφή του `ticket_type` για τη διάρκεια της συναλλαγής, εμποδίζοντας οποιοδήποτε άλλο ταυτόχρονο αίτημα από το να διαβάσει ή να τροποποιήσει τον ίδιο αριθμό διαθέσιμων εισιτηρίων.

Η ακολουθία ελέγχων εντός της συναλλαγής:

1. Lock του `ticket_type` row
2. Έλεγχος αν τα ζητούμενα εισιτήρια είναι διαθέσιμα
3. Έλεγχος ολικής χωρητικότητας εκδήλωσης
4. Δημιουργία εγγραφής κράτησης
5. Μείωση `available` στον `ticket_type`

7.2 Αυτόματη Ολοκλήρωση Εκδηλώσεων

Εκδηλώσεις που έχουν λήξει (το `end_datetime` τους είναι στο παρελθόν) αλλά παραμένουν σε κατάσταση `PUBLISHED` σημειώνονται αυτόματα ως `COMPLETED`. Αυτό γίνεται:

- Αμέσως κατά την εκκίνηση του `server`
- Κάθε 5 λεπτά αυτόματα

7.3 Επικύρωση Εκδηλώσεων

Κατά τη δημιουργία ή επεξεργασία εκδήλωσης ελέγχονται:

- Τουλάχιστον μία κατηγορία
- Τουλάχιστον ένας τύπος εισιτηρίου

- Χωρητικότητα > 0
- end_datetime > start_datetime
- Άθροισμα ποσοτήτων εισιτηρίων ≤ χωρητικότητα εκδήλωσης
- Τιμή εισιτηρίου ≥ 0, ποσότητα > 0

7.4 XML Export

Το XML που παράγεται ακολουθεί ακριβώς το DTD που ορίζεται στις προδιαγραφές της εργασίας. Το inline DTD συμπεριλαμβάνεται στο αρχείο εξαγωγής, ώστε να είναι αυτοτελές. Ειδικοί χαρακτήρες γίνονται escape σωστά (&, <, >, ", ').

7.5 Παρακολούθηση Προβολών

Κάθε φορά που συνδεδεμένος χρήστης επισκέπτεται τη σελίδα λεπτομερειών μιας εκδήλωσης, καταγράφεται μία εγγραφή στον πίνακα event_views με UPSERT — εάν υπάρχει ήδη, ενημερώνεται μόνο το viewed_at. Έτσι κάθε ζεύγος (χρήστης, εκδήλωση) εμφανίζεται μία μόνο φορά.

8. Αλγόριθμος Συστάσεων

8.1 Επισκόπηση

Υλοποιείται ο αλγόριθμος **Biased Matrix Factorization (BMF)** εξ ολοκλήρου από μηδενική βάση, χωρίς χρήση εξωτερικής βιβλιοθήκης. Το μοντέλο προβλέπει βαθμολογίες χρήστη-εκδήλωσης με τον τύπο:

$$\hat{r}(u, i) = \mu + b_u + b_i + p_u \cdot q_i$$

Όπου:

- μ : παγκόσμιος μέσος όρος βαθμολογιών
- b_u : bias χρήστη
- b_i : bias εκδήλωσης
- p_u : latent factor vector χρήστη (διάσταση 20)
- q_i : latent factor vector εκδήλωσης (διάσταση 20)

Η βελτιστοποίηση γίνεται με **Stochastic Gradient Descent (SGD)** για 50 epochs, με ρυθμό μάθησης 0.005 και regularization 0.02.

8.2 Δεδομένα Εκπαίδευσης

Το μοντέλο εκπαιδεύεται σε δύο πηγές δεδομένων:

Dataset (event_interest.csv): Περιέχει 13.672 ζεύγη αλληλεπιδράσεων χρηστών-εκδηλώσεων. Μετατρέπεται σε βαθμολογίες:

Σήμα	Βαθμολογία
<code>interested = 1</code>	5
Κανένα σήμα (implicit view)	2
<code>not_interested = 1</code>	Αγνοείται

Τα ρητά αρνητικά σήματα αγνοούνται, καθώς η απουσία ενδιαφέροντος δεν σημαίνει απαραίτητα αρνητική προτίμηση για το συγκεκριμένο σύστημα.

Live DB: Αλληλεπιδράσεις χρηστών της εφαρμογής:

Αλληλεπίδραση	Βαθμολογία
Επιβεβαιωμένη κράτηση	5
Προβολή εκδήλωσης	1

8.3 Διαχωρισμός Χώρων ID

Τα IDs χρηστών και εκδηλώσεων του dataset (π.χ. χρήστης 8949, εκδήλωση 1130067) και της live βάσης (1, 2, 3...) συνυπάρχουν στο ίδιο μοντέλο. Για αποφυγή συγκρούσεων, τα IDs της live βάσης μετατοπίζονται κατά **+10.000.000** πριν εισαχθούν στο μοντέλο. Οι συστάσεις επιστρέφονται πάντα ως IDs εκδηλώσεων της live βάσης (αφαιρείται η μετατόπιση).

8.4 Πρόγραμμα Εκπαίδευσης

- **Κατά την εκκίνηση:** Το μοντέλο εκπαιδεύεται αμέσως (dataset + τρέχουσες αλληλεπιδράσεις ΒΔ)
- **Κάθε ώρα:** Επανεκπαίδευση με νέα δεδομένα από τη ΒΔ (το dataset φορτώνεται από cache)

Το dataset διαβάζεται από δίσκο μία φορά κατά την πρώτη εκπαίδευση και αποθηκεύεται στη μνήμη. Κάθε επόμενη επανεκπαίδευση χρησιμοποιεί τα δεδομένα από cache, εκτελώντας μόνο νέα queries στη ΒΔ.

8.5 Προτάσεις για Νέους Χρήστες

- Εάν ο χρήστης έχει κρατήσεις -> συστάσεις βάσει κρατήσεων και προβολών
- Εάν ο χρήστης έχει μόνο προβολές -> συστάσεις βάσει προβολών
- Εάν ο χρήστης δεν έχει καμία αλληλεπίδραση -> δεν εμφανίζονται συστάσεις

9. Ασφάλεια

9.1 HTTPS

Ολόκληρη η επικοινωνία backend–frontend γίνεται μέσω **HTTPS** με TLS. Κατά την ανάπτυξη χρησιμοποιείται self-signed πιστοποιητικό που παράγεται τοπικά.

9.2 Helmet

Χρησιμοποιείται το middleware **Helmet** που ορίζει αυτόματα ασφαλή HTTP headers: - Content-Security-Policy - X-Frame-Options - X-Content-Type-Options - Strict-Transport-Security - κ.ά.

9.3 CORS

Το CORS είναι ρυθμισμένο ώστε να επιτρέπει αιτήματα μόνο από την καταχωρημένη διεύθυνση frontend (FRONTEND_URL στο .env). Όλα τα cross-origin αιτήματα από άλλες πηγές απορρίπτονται.

9.4 Rate Limiting

Τα endpoints σύνδεσης και εγγραφής προστατεύονται από rate limiter: μέγιστο **20 αιτήματα ανά 15 λεπτά** ανά IP. Αυτό αποτρέπει επιθέσεις brute-force σε κωδικούς χρηστών.

9.5 Εξουσιοδότηση σε Επίπεδο Πόρων

Επιπλέον του role-based ελέγχου, εφαρμόζεται έλεγχος ιδιοκτησίας σε επίπεδο πόρου:

- Ο Διοργανωτής μπορεί να επεξεργαστεί/δημοσιεύσει/ακυρώσει μόνο τις δικές του εκδηλώσεις
- Ο Συμμετέχων μπορεί να ακυρώσει μόνο τις δικές του κρατήσεις
- Ο Διαχειριστής δεν μπορεί να αναστείλει τον εαυτό του

9.6 Απόκρυψη Ευαίσθητων Δεδομένων

Το password_hash δεν επιστρέφεται ποτέ σε API response

Το afm εμφανίζεται μόνο στον ίδιο τον χρήστη ή στον Διαχειριστή

10. Εγκατάσταση και Εκτέλεση

10.1 Προαπαιτούμενα

- Node.js v20+
- PostgreSQL 14+
- Angular CLI (για το frontend)
- WSL/Ubuntu (συνιστάται για Windows)

10.2 Αυτόματη Εγκατάσταση

Από τον φάκελο project/ σε WSL:

```
chmod +x setup.sh  
./setup.sh
```

Το script εκτελείται ως κανονικός χρήστης (όχι sudo) και χρησιμοποιεί sudo εσωτερικά μόνο για PostgreSQL. Χειρίζεται όλα τα βήματα αυτόματα και παραλείπει όσα έχουν ήδη γίνει.

10.3 Χειροκίνητη Εγκατάσταση

1. Εγκατάσταση Node.js 20 μέσω nvm

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh |  
bash  
source ~/.bashrc && nvm install 20 && nvm use 20
```

2. Εγκατάσταση PostgreSQL

```
sudo apt install -y postgresql postgresql-client  
sudo service postgresql start  
sudo -u postgres psql -c "ALTER USER postgres PASSWORD 'postgres123';"  
sudo -u postgres psql -c "CREATE DATABASE eventmanagement;"
```

3. Ρύθμιση backend

```
cd project/backend  
npm install  
cp .env.example .env # επεξεργασία .env με τις σωστές τιμές
```

4. Migrations και seed

```
npx prisma migrate deploy  
npx prisma db seed
```

5. SSL πιστοποιητικά

```
mkdir -p certs  
openssl req -x509 -newkey rsa:2048 -keyout certs/key.pem -out certs/cert.pem  
-days 365 -nodes -subj "/CN=localhost"
```

10.4 Εκτέλεση

Terminal 1 – Backend

```
sudo service postgresql start  
cd project/backend  
npm run dev
```

Terminal 2 – Frontend

```
cd project/frontend  
npm start
```

10.5 Αποδοχή SSL Πιστοποιητικού

Κατά την πρώτη εκκίνηση (ή μετά από επανεκκίνηση browser), ανοίξτε:

<https://localhost:3000/api/health>

Κλικ **Advanced** → **Proceed to localhost**. Μετά επισκεφθείτε <http://localhost:4200>.

Default admin: admin / admin``123

10.6 Μεταβλητές Περιβάλλοντος (.env)

Μεταβλητή	Περιγραφή
DATABASE_URL	PostgreSQL connection string
JWT_SECRET	Τυχαίο μυστικό για υπογραφή JWT
JWT_EXPIRES_IN	Διάρκεια ζωής token (π.χ. 24h)
PORT	Θύρα backend (default: 3000)
FRONTEND_URL	URL frontend για CORS
SSL_KEY_PATH	Διαδρομή private key
SSL_CERT_PATH	Διαδρομή πιστοποιητικού
DATASET_PATH	Διαδρομή event_interest.csv (προαιρετικό)

11. Επίλογος

11.1 Σύνοψη Υλοποίησης

Κατά την ανάπτυξη του backend υλοποιήθηκαν:

- Πλήρης REST API με Express και Typescript
- Σχεδιασμός και υλοποίηση σχεσιακής βάσης 8 πινάκων με Prisma ORM και versioned migrations
- Σύστημα αυθεντικοποίησης JWT με live έλεγχο κατάστασης χρήστη
- Ασφαλής κράτηση εισιτηρίων με row-level locking για αποφυγή race conditions
- Αυτόματη μετάβαση εκδηλώσεων σε COMPLETED κατάσταση
- Σύστημα μηνυμάτων με soft delete και σελιδοποίηση
- Export εκδηλώσεων σε XML (με inline DTD) και JSON
- Αλγόριθμος Biased Matrix Factorization υλοποιημένος από μηδενική βάση
- Μηχανισμός ασφαλείας (HTTPS, Helmet, CORS, rate limiting)

11.2 Δυσκολίες και Αντιμετώπισή τους

Μετάβαση από raw SQL σε Prisma ORM: Αρχικά το backend χρησιμοποιούσε απευθείας queries μέσω της βιβλιοθήκης pg. Η μετάβαση στο Prisma απαίτησε επανασχεδιασμό των queries, προσθήκη `@map` directives για σωστή αντιστοίχιση με τα ονόματα πινάκων της PostgreSQL, και επανεκτέλεση του `npx prisma generate`.

Row-level locking με Prisma: Το Prisma δεν υποστηρίζει `SELECT ... FOR UPDATE` στο standard API. Η λύση ήταν η χρήση `prisma .queryRaw` εντός `prisma .transaction`, που επιτρέπει την εκτέλεση raw SQL μέσα σε διαχειριζόμενη συναλλαγή.

Αποδοχή self-signed πιστοποιητικού: Ο browser δεν αποθηκεύει μόνιμα την αποδοχή για localhost. Απαιτείται εκ νέου αποδοχή μετά από επανεκκίνηση του browser. Αυτό είναι αναμενόμενη συμπεριφορά για development περιβάλλον.

Ενσωμάτωση dataset στον αλγόριθμο συστάσεων: Τα IDs χρηστών και εκδηλώσεων του παρεχόμενου dataset και της live βάσης βρίσκονται στον ίδιο αριθμητικό χώρο και μπορούν να συγκρουστούν. Η λύση ήταν η εφαρμογή offset +10.000.000 στα IDs της live βάσης πριν την εκπαίδευση του μοντέλου.